

Soum Deco

Smarter Solutions Within the Project's Limits

A second pass through the audit findings, this time respecting the explicit design constraints the project was built around: Cloudflare free tier only, Apps Script 20K calls/day, KV 1K writes/day, Cloudinary free 25GB/25GB, no credit card on file, Algerian 3G mobile target, and a 9,500-product aspiration. The first report's recommendations were correct but some assumed the team could pay for services — this addendum proposes alternatives that fit the original budget: zero dollars, zero new accounts, zero credit cards.

REFRAMING

The first report said things like 'add Cloudflare Image Resizing (\$5/mo)', 'use Cloudflare Access service tokens', 'add Cloudflare Turnstile CAPTCHA'. Those recommendations are technically correct but they violate the project's **explicit hard constraint of zero paid services and no credit card**. The wrangler.toml file literally says 'R2 requires a credit card to enable (even though the free tier covers 10 GB)' — that constraint is non-negotiable. Every fix in this addendum works within that constraint. The price: slightly more engineering work, and one or two creative uses of free-tier services that the team already has access to.

8

Constraints honored

18

Recs revised

0

\$ added cost

0

New accounts needed

0

Credit cards needed

Project	soumdeco-magic-v3-final
Companion to	SoumDeco_Audit_Report.pdf (the brutal-honesty audit)
Constraint	Cloudflare free + Apps Script free + Cloudinary free + no CC

Target	Algerian COD e-commerce, 3G mobile, 9,500-product aspiration
Addendum date	2026-08-19
Position	Not a contradiction — a refinement

1. The Constraints the Project Designed Around

Before proposing anything, I had to re-extract the explicit design constraints from the code itself. The team documented these in code comments across 8 files, and they explain almost every architectural choice that initially looked weird. None of these are guesses — they are stated in the source.

1.1 The 8 hard constraints (from source)

#	Constraint	Source — code comment	Why it exists
C1	No credit card on Cloudflare account	wrangler.toml:25-27: "R2 requires a credit card to enable (even though the free tier covers 10 GB)"	Blocks R2, Image Resizing, Cloudflare Access paid tier, Workers Paid plan, Durable Objects, Queues, Hyperdrive. Anything that needs billing setup is OUT.
C2	Apps Script quota: 20,000 calls/day	use-catalog.ts:120: "can handle 50K+ visits/day without hitting Apps Script's 20K/day limit"	Each /refresh, each ?action=products, each ?action=stock, each ?action=order counts. 50K visitors/day cannot call Apps Script directly — must go through the Worker cache.
C3	KV writes: 1,000/day (free tier)	worker/data-sync.js:519-521: "unsampled = 50K writes/day (50× the 1K/day quota). Sampled at 1-in-50 = ~1K writes/day"	CRITICAL: each /refresh = 4 KV writes. Burst of 250 /refresh calls = quota exhausted. This is why the 1-sec rate limit exists — it's protecting against KV write quota, not DOS.
C4	Worker requests: 100,000/day (free)	worker/README.md:60: "Worker requests: ~50K / 100K / 50%"	Designed for ~50K visitors/day. Already at 50% headroom. Polling 1-min × 50K visitors × 4 min avg session = ~200K requests/day → BLOWS THE QUOTA at the target scale.
C5	Cloudinary free: 25GB storage + 25GB bandwidth/month	drive-upload.ts:5-8: "Cloudinary free tier: 25GB storage, 25GB bandwidth/month"	At ~70KB per image: 357K image-loads/month = ~12K unique visitors/month if every visitor loads 30 images. Throttles hard when 80+ images load simultaneously (the source of the 'images don't show' bug).
C6	Cloudflare Pages: 20,000 files per project	scripts/auto-sync.py:14,37: "CF_PAGES_FILE_LIMIT = 20_000" + SAFE_FILE_LIMIT = 19_000	At 9,500 products × 5 images = 47,500 files (2.4× over the limit). This is why the manifest strategy was invented — but it can't actually scale to 9,500 products without breaking Pages' build.
C7	Target: Algerian 3G mobile users	(inferred) layout.tsx + Globals.css RTL Arabic + Algiers/Algeria in brand-config	Connection: 400-800 kbps, 200-400ms RTT, ~\$0.50/GB data cost. Bundle size + image bytes have outsized UX impact. Service worker was tried + unregistered (conflict).
C8	localStorage 5MB limit per origin	adaptive-storage.ts:4-9: "For large catalogs (9,500 products × 2 KB each = 19 MB), it overflows"	Catalogs over 4MB auto-fallback to IndexedDB. But the JSON.parse + JSON.stringify still block the main thread. The fallback handles storage, not performance.

What this means in practice

Every fix proposed in this addendum has to clear ALL 8 constraints simultaneously. A fix that solves the Cloudinary bug but adds a \$5/mo service is invalid. A fix that stops the data leak but triples KV writes is invalid. A fix that adds 200KB to the JS bundle is invalid for 3G mobile users. The original audit's recommendations were engineering-correct but constraint-blind — this addendum corrects that without compromising the security or data-integrity conclusions.

2. Recommendations to Reconsider

Of the ~50 recommendations in the original report, 18 need to be reconsidered because they implicitly assume a paid service or an account upgrade. This section goes through each one and proposes a constraint-respecting alternative. The original findings stand — only the proposed fixes change.

2.1 Comparison: original recommendation vs smarter alternative

Concern	Original recommendation	Why it breaks the project's limits	Smarter alternative (constraint-respecting)
Auth — admin login	POST /api/admin/login + HTTP-only session cookie + signed JWT (Pages API).	Pages API uses edge runtime which is broken on next-on-pages v1 (the team's own comment in client-sheet.ts:7-8). Signed JWT needs a secret-management pattern that doesn't leak.	Use the EXISTING Worker /refresh secret as a bearer token. Admin panel sends it as X-Admin-Secret on all mutating calls. Worker validates. No new infra, no JWT library, no session table. Reuses a secret that's already deployed.
Auth — admin writes (Apps Script)	Route admin writes through authenticated /api/products Pages routes.	Pages API routes are dead code (admin panel bypasses them). Cloudflare edge runtime is broken. Adding middleware.ts requires moving off @cloudflare/next-on-pages or upgrading to Workers Paid.	Add a tiny 'auth gate' endpoint to the EXISTING Worker: POST /?action=write-product&secret;=X. Worker forwards to Apps Script with the secret stripped. Admin panel posts to Worker, not directly to Apps Script. Zero new infra — just a new branch in data-sync.js.
Auth — order rate limiting	Cloudflare Turnstile CAPTCHA on order form.	Turnstile is free but requires a site key + secret + signup. Adds a script tag + ~30KB to the order form. Works but adds friction on Algerian mobile.	Honeypot field + time-to-submit check + per-wilaya-per-hour KV counter. The Worker already has KV access. 3 lines of code in worker/data-sync.js + a hidden input in cod-order-form.tsx. Zero new dependencies, zero new accounts.
Image upload — Cloudinary signed uploads	Convert preset to signed + sign on server with API secret.	Server = Pages edge runtime = broken. No way to safely store the Cloudinary API secret in client. Worker could sign but adds complexity.	Keep unsigned preset BUT add a Worker-side proxy: POST /?action=upload-image&secret;=X → Worker forwards to Cloudinary with the file. Cloudinary preset stays unsigned (simpler). Worker becomes the only path to upload. Limits abuse to people who have the admin secret (which we've now made server-side only).
Image optimization — srcset / responsive images	Enable Cloudflare Image Resizing (\$5/mo) OR generate WebP variants at build time.	Image Resizing requires paid plan (C1). Build-time variants double/triple the file count (C6: 20K file limit). At 9,500 products × 5 images × 3 widths = 142,500 files = 7 × OVER the limit.	Use Cloudinary's free transformation API (already deployed). For LOCAL images (post-sync), accept the single-resolution WebP at 850px max. Add a tiny with srcset only when the image is still on Cloudinary (using its /c_limit,w_400,w_800,w_1200/transform). Cost: zero. Image quality hit: minimal (850px is sharp on most phones).

Concern	Original recommendation	Why it breaks the project's limits	Smarter alternative (constraint-respecting)
PII transmission — POST body instead of GET	Move order submission from GET to POST (Apps Script doPost).	Apps Script's doPost is flaky — the codebase already discovered this (client-sheet.ts:197: '302 → 405 after redirect, but the product IS saved before the redirect'). Switching to POST for orders risks the same flakiness.	Keep GET but encrypt the PII payload with a simple XOR + base64 using a per-order nonce stored in the URL hash (not query string). Apps Script decodes. Or, simpler: encode PII as base64 in the URL (not encrypted but at least not human-readable in logs). The real fix is moving the order endpoint to the Worker (which already has KV + fetch). Worker POSTs body to Apps Script server-side. Browser only sends PII to Worker.
Rate limiting /api/order	Cloudflare KV-based per-IP rate limiting OR Cloudflare Turnstile.	Turnstile needs signup (mild C1 violation). KV-based per-IP rate limit would burn 1 KV write per order = 50K writes/day at scale = 50× OVER the 1K/day quota (C3).	Use a probabilistic rate limiter: 1-in-10 orders write to KV (sampled, like the hit counter at data-sync.js:527). Plus in-memory rate limit per Worker isolate (Cloudflare Workers have a global cache that survives within an isolate). Plus per-phone-prefix check using the Apps Script sheet itself (read existing orders for that phone in last 1h, reject if > 3).
Auth — worker /refresh secret rotation	Stop using NEXT_PUBLIC_WORKER_ADMIN_SECRET, use server-side WORKER_ADMIN_SECRET.	Pages edge runtime is broken. There is no 'server-side' that isn't edge. Cannot use Node runtime on Cloudflare Pages.	Move the /refresh caller to the Worker itself. Admin panel calls Worker /?action=refresh (already does). Worker validates against env.ADMIN_SECRET (already does, worker/wrangler.toml secret). Remove NEXT_PUBLIC_WORKER_ADMIN_SECRET entirely — admin panel doesn't need it because Worker handles the Apps Script call. Browser→Worker is fine because the Worker is the secret-keeper.
Backup strategy — Google Sheet only copy	Add automated backups to a second Google Sheet + Cloudflare R2.	R2 needs credit card (C1). Second Sheet is feasible but adds ops burden.	Use the EXISTING Worker to write a daily snapshot to KV (key: snapshot_YYYY_MM_DD, TTL: 30 days). KV value cap is 25MB — fits a 9,500-product catalog if compressed. Cost: 30 KV writes/month = 0.03% of quota. Plus: export Apps Script sheet → CSV → commit to git via GitHub Action (already runs auto-sync.py daily). Git IS the backup.
Catalog at scale (9,500 products) — virtualization	Add @tanstack/react-virtual (already in deps) for DOM virtualization.	VALID — this recommendation already fits constraints. Just confirming it.	(No change needed. Already constraint-correct.)

Concern	Original recommendation	Why it breaks the project's limits	Smarter alternative (constraint-respecting)
Catalog at scale — pagination	Server-side pagination via <code>/api/products?page=N</code> .	Pages API is dead code + edge runtime broken. Apps Script can't paginate efficiently.	Move pagination to the Worker. Worker already has the full catalog in KV. Add <code>?action=catalog&page=N&pageSize=48&category=X</code> . Worker slices the JSON in-memory. Browser fetches 1 page at a time. $9,500 \text{ products} \div 48/\text{page} = 198 \text{ pages}$. Initial load = 1 page = ~5KB. Total catalog still in KV (1 read) — no quota impact.
Polling — reduce <code>POLL_MS</code> from 60s to 300s	Increase poll interval to 5 min.	VALID. Already constraint-correct.	(No change needed. But add: Server-Sent Events from Worker when admin makes a change. Worker already writes to KV on <code>/refresh</code> — it could ALSO broadcast to all connected clients via a 'changes' KV key with a timestamp. Clients poll that tiny key every 30s instead of the full catalog. 1 KV read = 1 byte. Quota impact: ~0.)
Image upload — replace unsigned Cloudinary with R2	Enable R2 + use <code>/api/r2-upload</code> .	R2 needs credit card (C1, explicitly documented).	Stay on Cloudinary BUT route uploads through the Worker (which validates the admin secret). Cloudinary preset stays unsigned. Worker is the only path that knows the admin secret. Browser → Worker → Cloudinary. Adds 1 hop but solves the 'anyone can upload' problem without enabling R2.
Service worker (PWA / offline support)	Add a service worker for offline catalog + image caching.	The team tried this and explicitly unregistered it (public/unregister-sw.js + comment 'was causing stuck at loading'). Service workers on Cloudflare Pages with <code>@cloudflare/next-on-pages</code> v1 have known issues.	Use IndexedDB as the offline cache (already implemented via <code>adaptive-storage.ts</code>). Add a tiny 'cache-first' wrapper around <code>fetchCatalog</code> that prefers IDB if it's <5min old. NO service worker needed. Already 80% implemented.
CSP headers	Add Content-Security-Policy in <code>next.config.ts</code> .	<code>next.config.ts</code> is the right place, but <code>@cloudflare/next-on-pages</code> strips custom headers in some configurations. Need to verify they actually ship.	Add CSP via <code>/public/_headers</code> (Cloudflare Pages native — already used for caching rules). This is the canonical CF Pages way and bypasses <code>next.config</code> entirely. Example: <code>'Content-Security-Policy: default-src \'self\'; img-src \'self\' https://res.cloudinary.com; ...'</code>
Tests — Vitest + Playwright	Add Vitest unit tests + Playwright e2e.	Playwright needs a browser install (~300MB) in CI. GitHub Actions free tier has 2K min/month for Linux runners. Playwright adds ~5 min per run.	Vitest: yes, add. Playwright: only for the critical paths (admin auth flow, order submission, image upload). 3-5 tests max. Run only on PRs to main, not on every push. Saves CI minutes.

Concern	Original recommendation	Why it breaks the project's limits	Smarter alternative (constraint-respecting)
CI/CD pipeline	Add GitHub Actions workflow.	GitHub Actions free tier: 2,000 min/month for private repos, unlimited for public. This repo appears private.	Make the repo PUBLIC (it's just a Next.js storefront, no proprietary code). Unlimited CI minutes. Add: lint + typecheck + build on PR, auto-sync.py on schedule, deploy on merge to main. All free.
Database — Prisma + SQLite exists but unused	Either use Prisma or remove it.	Prisma doesn't work on Cloudflare Pages edge runtime. SQLite needs a server. Removing it is the right call.	Remove Prisma from package.json (already optionalDependencies). Delete /prisma/schema.prisma. The 'database' is the Google Sheet — that's the right choice for this constraint set. Document this decision in the README.

3. The Cloudinary Bug — Constraint-Aware Fix

The original report identified three root causes for the Cloudinary image bug: (1) local URL leak into the Google Sheet, (2) hardcoded paths in build-image-manifest.py, (3) 24h sync lag. The proposed fixes were correct, but I missed one critical constraint-respecting option that solves all three at once with one architectural change. Plus, the original 'fix #5' (remove URL rewriting entirely, serve from Cloudinary) would have violated the 25GB/month bandwidth constraint at scale. Here's the smarter version.

3.1 The single architectural change that fixes the leak

The leak happens because the frontend rewrites Cloudinary URLs to local paths **on the React state object**, and that state gets persisted back to Apps Script on the next admin save. The fix is to NEVER rewrite the URL on the state — instead, rewrite it **at render time only**, in the ProductImage component. The product object always stores the canonical Cloudinary URL; the rendering layer decides where to fetch the bytes from. This is a 30-line refactor that touches 2 files and solves the leak completely without any data migration.

```
// BEFORE (use-catalog.ts:797-825 – the leak)
function optimizeCloudinaryUrls(p: Product): Product {
  // rewrites p.image and p.images[] from Cloudinary → /images/products/...
  // returns a NEW product object with rewritten URLs
  // ▲ This new object is what gets saved to sheet on next admin save
}

// AFTER (use-catalog.ts:797-825 – no-op, deleted)
// The Product object ALWAYS stores the canonical Cloudinary URL.
// The manifest lookup happens at render time, in ProductImage.

// product-image.tsx (the new logic – add ~20 lines)
function pickImageUrl(cloudinaryUrl: string, size: 'card'|'full'): string {
  // 1. Extract filename from Cloudinary URL
  const fn = extractFilename(cloudinaryUrl);
  if (!fn) return cloudinaryUrl; // not Cloudinary, return as-is

  // 2. Check the manifest (already loaded – no new fetch)
  const localPath = getLocalPathSync(cloudinaryUrl);
  if (localPath) return localPath; // serve from Pages (free, unlimited)

  // 3. Not yet synced – serve from Cloudinary with optimization params
  const w = size === 'full' ? 'w_800' : 'w_400';
  return cloudinaryUrl.replace(
    '/image/upload/',
    `/image/upload/c_limit,${w},q_auto,f_auto/`
  );
}

// ProductImage uses pickImageUrl(src, size) instead of optimizeImageUrl(src, size).
// The onError fallback becomes unnecessary (Cloudinary URL is always valid).
```

Why this is better than the original fix: The original fix proposed storing the original Cloudinary URL in a separate field (imageOriginal) and round-tripping it before save. That works but adds state complexity and a migration step. This approach removes the leak at the source — the Product object is never mutated, so there's nothing to round-trip. The existing 44 corrupted sheet rows still need a one-time repair (original fix #2 stands), but no NEW corruption can happen.

3.2 The build-image-manifest.py fix — already correct, just verifying

The original fix (use SCRIPT_DIR/REPO_ROOT pattern) is correct and constraint-safe. No changes needed. But there's an additional improvement: make the script fail loudly if the output manifest is empty. Currently it silently writes an empty manifest, which is what caused the original bug. Add a check: if `len(files) == 0`, exit with a non-zero status and print an error. The build will fail, but that's better than deploying with an empty manifest and breaking every image on the site.

```
# scripts/build-image-manifest.py – add at the end of main()
if len(files) == 0:
    print("ERROR: No image files found in /public/images/products/")
    print("This will break all Cloudinary images on the site.")
    print("Either add images OR disable the local-image optimization.")
    sys.exit(1) # FAIL THE BUILD – better than silent corruption

print(f"Manifest: {len(files)} files (Pages limit: 20,000)")
```

3.3 The sync interval — smarter than 'reduce to 6h'

The original fix said reduce auto-sync from 24h to 6h. That's correct but blunt. The smarter version: trigger auto-sync **only when there's something new to sync**. The admin panel already calls Worker /refresh after every image upload. The Worker could ALSO trigger a GitHub Actions workflow_dispatch via a webhook. New images sync within minutes of upload, not on a fixed schedule. The 24h schedule stays as a safety net for missed triggers.

```
# 1. Add a GitHub Actions workflow_dispatch endpoint
# .github/workflows/sync-images.yml (new file)
on:
  schedule: [{cron: '0 0 * * *'}] # daily safety net
  workflow_dispatch: # manual + webhook trigger
inputs:
  trigger_reason:
    description: 'Why this sync was triggered'
    required: false

# 2. Worker /refresh endpoint, after KV write, fires webhook
# worker/data-sync.js (in syncData, after successful KV writes)
if (productsChanged && env.GH_ACTIONS_WEBHOOK_URL) {
  // Fire-and-forget – non-blocking
  ctx.waitUntil(
    fetch(env.GH_ACTIONS_WEBHOOK_URL, {
      method: 'POST',
      headers: {'Authorization': `Bearer ${env.GH_ACTIONS_TOKEN}`},
      body: JSON.stringify({ref: 'main', inputs: {trigger_reason: 'admin_op'}})
    }).catch(() => {})
  );
}
```

3.4 Bandwidth math — staying under 25GB/month on Cloudinary free

The original 'fix #5' (remove URL rewriting, serve all from Cloudinary) would burn the 25GB/month Cloudinary free tier at ~12K visitors/month (assuming 30 images $\times$ ~70KB each per visitor). That's not viable. The local-mirror strategy **MUST** stay. But there's a smarter version of it: serve the WebP-optimized version from Cloudinary **DURING** the sync lag (1-7 days), then switch to the local file **AFTER** sync. The current code does this — but the leak bug breaks the switch. Fix the leak (3.1) and the bandwidth math works: ~5KB/image from Cloudinary (small w_400 WebP) for ~24h $\times$ ~100 new images/day $\times$ ~5 visitors/day each = 2.5MB/day = 75MB/month. Well under the 25GB limit. Pages serves the other 99% of traffic for free.

Cloudinary bandwidth budget (corrected)

After fixing the leak with the architectural change in 3.1: **~75MB/month** on Cloudinary free tier (25GB limit = 0.3% used). The local-mirror strategy is correct and stays. The leak was the only thing wrong with it.

4. The Auth Problem — Solved with Existing Infrastructure

The original report correctly identified that there is no real authentication anywhere. But its proposed solution (HTTP-only cookies + signed JWT + Cloudflare Access) requires either paid services or a working Pages API (which is broken on the team's chosen deployment). The smarter approach: **the Worker already has a working secret-based auth system for /refresh — extend it to cover all mutating operations.** Zero new infra, zero new dependencies, zero new accounts.

4.1 The auth architecture (using only what's already deployed)



4.2 What changes (concrete diffs)

Change 1: Move admin writes from Apps Script to Worker

```
# worker/data-sync.js – add new branches in the fetch() handler

if (action === 'write_product') {
  // 1. Validate admin secret (same pattern as /refresh)
  const secret = request.headers.get('X-Admin-Secret') || '';
  if (secret !== env.ADMIN_SECRET) {
    return json({ok: false, error: 'unauthorized'}, 401, cors);
  }
  // 2. Forward to Apps Script (server-side, no CORS, no browser)
  const body = await request.text();
  const res = await fetch(`${env.APPS_SCRIPT_URL}?action=product_create`, {
    method: 'POST',
    headers: {'Content-Type': 'text/plain;charset=utf-8'},
    body: body,
    redirect: 'follow',
  });
  // 3. Trigger KV refresh (existing function)
  ctx.waitUntil(syncData(env).catch(() => {}));
  return json({ok: res.ok || res.status === 302 || res.status === 405}, 200, cors);
}

# Same pattern for: delete_product, reset_products, upload_image (proxy to Cloudinary)
```

Change 2: Remove direct browser → Apps Script calls

```
# src/lib/sheet.ts – DELETE the hardcoded URL
# (currently line 8-9 – this is what makes Apps Script publicly reachable)

# src/lib/client-sheet.ts – replace clientUpsertProduct etc.
# Point them at the Worker instead of Apps Script

export async function clientUpsertProduct(product: SheetProduct): Promise<boolean> {
  const workerUrl = process.env.NEXT_PUBLIC_WORKER_URL;
  const secret = sessionStorage.getItem('soundeco_admin_secret');
  // ^ secret is in sessionStorage, NOT in the JS bundle.
  // It's set by the admin login flow (Change 3 below).

  const res = await fetch(`${workerUrl}?action=write_product`, {
    method: 'POST',
    headers: {
      'X-Admin-Secret': secret || '',
      'Content-Type': 'text/plain;charset=utf-8',
    },
    body: JSON.stringify(product),
  });
  return res.ok;
}
```

Change 3: Admin login flow (no JWT, no sessions table)

```
# src/components/site/admin-panel.tsx – PasswordGate
# Instead of comparing to a hardcoded password, ask the Worker to validate.

const submit = async () => {
  const res = await fetch(`${WORKER_URL}/?action=auth`, {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({password: value}),
  });
  const data = await res.json();
  if (data.ok && data.token) {
    sessionStorage.setItem('soundeco_admin_secret', data.token);
    onAuthed();
  } else {
    setError(true);
  }
};

# Worker validates the password (which lives in env.ADMIN_PASSWORD – set via
# wrangler secret put, NOT in the bundle) and returns the ADMIN_SECRET as a
# 'token'. The token is the same env.ADMIN_SECRET the Worker already uses.
# No JWT, no session table, no new infra. The 'session' is just sessionStorage.
```

4.3 Why this is constraint-safe

Constraint	Status
C1 — No credit card	✓ Worker is on free tier. Worker /refresh already works. Adding more endpoints doesn't change the plan.
C2 — Apps Script 20K/day	✓ Apps Script is now ONLY callable from the Worker. Cron = 1,440 calls/day. Admin writes = ~50/day max. Total: ~1,500/day = 7.5% of quota.
C3 — KV writes 1K/day	✓ Each admin write triggers syncData which writes 4 KV keys. ~50 admin writes/day × 4 = 200 KV writes/day = 20% of quota. Safe.
C4 — Worker requests 100K/day	✓ Admin requests add ~50/day to the existing 50K visitor reads. Negligible.
C5 — Cloudinary 25GB/month	✓ Image upload proxy adds 0 bytes to Cloudinary (just forwards).
C6 — Pages 20K files	✓ No change to file count.
C7 — 3G mobile	✓ No additional JS bundle weight. sessionStorage lookup is instant.
C8 — localStorage 5MB	✓ sessionStorage used for token only (~64 bytes).

5. Catalog at Scale — The 9,500-Product Problem

The original report correctly flagged that the catalog will crash mobile browsers at 9,500 products (~57,000 DOM nodes). But its proposed fix (virtualization + pagination) left two unaddressed constraints: **(C6) the 20,000-file limit on Cloudflare Pages** means the local-image mirror can't hold $9,500 \times 5 = 47,500$ images, and **(C2) the Apps Script 20K calls/day limit** means the catalog can't be re-fetched per page. The team's aspiration to 9,500 products is the single hardest constraint to satisfy. Here's the smarter path.

5.1 The hard math of 9,500 products

```
Products: 9,500
Images/product: 5 (max)
Total images: 47,500

Cloudflare Pages file limit: 20,000 ← OVER by 27,500
Local image mirror max capacity: 19,000 ← safe headroom

Catalog JSON size (estimate):
Per product: ~2 KB (name, desc, category, price, images URLs, variations)
Total: 9,500 × 2 KB = 19 MB

Storage limits:
localStorage: 5 MB ← OVER by 14 MB (auto-falls back to IndexedDB)
IndexedDB: 50 MB+ ← OK (uses ~40% of typical quota)
KV value cap: 25 MB ← OK (just barely)

Worker quota impact:
KV read: 1 per visitor × 50K visitors = 50K reads/day (50% of quota) ✓
KV write: ~50/day on change (5% of quota) ✓
Worker CPU: 1 JSON.parse + 1 JSON.stringify per request
At 19 MB JSON: parse takes ~200ms on Worker isolate
Worker CPU limit: 10ms/request on free tier ← BLOWS THE QUOTA
```

The killer constraint is **Worker CPU time**. The free tier allows 10ms of CPU per request. Parsing a 19MB JSON string takes ~200ms. The Worker will hit the CPU limit on EVERY catalog request at scale. This is the constraint the original report missed — and it's the one that forces a different architecture.

5.2 The constraint-respecting architecture for scale

The answer is **sharded KV**. Instead of one giant catalog JSON, store the catalog as N shards (e.g., 26 shards keyed by first letter of product ID, or 100 shards keyed by hash). Worker fetches only the shards the visitor needs (e.g., the first 48 products of the first category). KV reads stay flat, but each read returns ~200KB instead of 19MB. Worker CPU drops to ~2ms per request. The 1K writes/day quota is unchanged (each sync still writes the same N keys, just smaller).

```

# Worker KV schema (new – replaces single 'products' key)
# Keys:
# shard_00 → JSON[Product] (first 100 products by sortOrder)
# shard_01 → JSON[Product] (next 100)
# ...
# shard_95 → JSON[Product] (last 100)
# __meta → {shardCount, productCount, lastSync, ...}
# __index → {productId → shardNumber, positionInShard}

# Visitor request: GET ?action=catalog&page=1&pageSize=48
# Worker:
# 1. Read __meta (1 KV read, ~200 bytes)
# 2. Determine which shard(s) to read for page 1 (1 KV read, ~200KB)
# 3. Return sliced + sorted result
# Total: 2 KV reads = 2ms CPU = under 10ms limit ✓

# Sync (cron):
# 1. Fetch full catalog from Apps Script (1 call, 19MB response)
# 2. Split into 96 shards
# 3. Write 96 + 2 (meta + index) = 98 KV writes per sync
# 4. Cron every 5 min = 288 syncs/day × 98 writes = 28,224 writes/day
# ⚠ OVER the 1K/day quota by 28× – need to change sync strategy

# Smart sync: only write shards that CHANGED (compare hash)
# Typical: ~5 shards change per day = 5 writes + 1 meta = 6 writes/sync
# Cron every 5 min × 288 syncs × 6 writes = 1,728 writes/day
# Still slightly over – reduce cron to every 10 min = 864 writes/day ✓

```

5.3 The image file count problem (C6)

9,500 products × 5 images = 47,500 files. Cloudflare Pages limit = 20,000. This is a hard wall. The local-mirror strategy CANNOT scale to 9,500 products. Three options:

Option	How it works	Constraint impact	Verdict
A. Limit local mirror to top N products	Only sync images for the 3,000 most-viewed products (15K files, under 20K limit). Others stay on Cloudinary.	C5: Cloudinary bandwidth rises. ~6,500 products × 5 images × 5KB (WebP w_400) × ~10 visitors/day = 1.6GB/day = 48GB/month = 1.9× OVER Cloudinary free.	BLOCKED by C5
B. Use Cloudinary's free image CDN only	Don't mirror to Pages at all. Serve all images from Cloudinary with f_auto,q_auto,w_400.	C5: 9,500 × 5 × 5KB × 10 visitors/day = 2.4GB/day = 72GB/month = 2.9× OVER Cloudinary free.	BLOCKED by C5
C. Add a second free Cloudflare Pages project for images	Create soumdeco-images.pages.dev. Pages are separate projects, each with 20K file limit. Total: 40K images across 2 projects.	All constraints satisfied. Adds 1 deploy step to auto-sync.py. Free.	✓ RECOMMENDED
D. Use jsDelivr CDN (free, unlimited)	Push images to a GitHub release. jsDelivr serves them free, unlimited bandwidth, global CDN.	All constraints satisfied. Adds GitHub release step. Free. But each release has a 2GB file limit (can use multiple).	✓ Alternative
E. Drop the 9,500-product target	Cap catalog at 3,000 products (15K images, under 20K file limit). Most boutique stores have <1K SKUs anyway.	All constraints satisfied. Business decision, not technical.	✓ Business option

Recommendation: Option C (second Pages project). It's the only one that fully satisfies all constraints AND scales to 9,500 products. The cost is one extra deploy step in auto-sync.py (push to soumdeco-images.pages.dev after pushing to the main project). Option E (cap at 3,000 products) is the business-level fallback if the team decides the 9,500 aspiration isn't worth the engineering complexity.

6. Polling — The Quota-Aware Strategy

The original report said 'increase POLL_MS from 60s to 300s.' That's correct but blunt. The deeper issue is that the polling strategy fights against the Worker's 100K requests/day quota (C4). At 50K visitors $\times$ 4-min avg session $\times$ 1 poll/min = 200K requests/day = 2 $\times$ OVER the quota. Here's the smarter version that gets near-instant updates without blowing any quota.

6.1 The insight: most polls return unchanged data

The catalog changes maybe 5-10 times per day (admin edits). The other 200,000 polls per day return the exact same data. The Worker already computes a hash for change detection (worker/data-sync.js:73-80, hashString function). The smarter strategy: poll a TINY 'version' key (8 bytes) instead of the full catalog. If the version changed, then fetch the full catalog. If not, skip. This drops Worker KV reads from 50K/day of 200KB each to 50K/day of 8 bytes each — same read count, but the Worker CPU drops from ~5ms per request to ~0.5ms per request.

```
# Worker: new endpoint ?action=version
# Returns just the hash + timestamp (8 bytes total)
if (action === 'version') {
  const meta = await env.CATALOG_KV.get('__meta', 'json');
  return json({
    v: meta?.lastChange || 0,
    h: meta?.productsHash || '0', // already computed in syncData
  }, 200, cors);
}

# Browser: poll version every 30s (cheap),
# fetch full catalog only when version changes
let lastVersion = 0;
setInterval(async () => {
  const v = await fetch(`${WORKER}?action=version`).then(r => r.json());
  if (v.v !== lastVersion) {
    lastVersion = v.v;
    refresh(); // full catalog fetch — only on change
  }
}, 30_000); // 30s polling, but most polls are 8-byte responses
```

6.2 The math: how this fits the quota

Strategy	Worker req/day	KV reads/day	Worker CPU/day	Quota fit
Current (1-min full poll)	200K	200K $\times$ 200KB	200K $\times$ 5ms = 1000s	2 $\times$ OVER req, 2 $\times$ OVER reads, CPU ok
Original fix (5-min full poll)	50K	50K $\times$ 200KB	50K $\times$ 5ms = 250s	OK req (50%), OK reads (50%), CPU ok
Smarter (30s version + on-change full)	150K version polls + ~10 full fetches	150K $\times$ 8B + 10 $\times$ 200KB	150K $\times$ 0.5ms + 10 $\times$ 5ms = 80s	1.5 $\times$ OVER req, OK reads, OK CPU
Smarter + visible-tab only (5-min version)	50K version polls + ~10 full	50K $\times$ 8B + 10 $\times$ 200KB	50K $\times$ 0.5ms + 10 $\times$ 5ms = 30s	OK req (50%), OK reads (50%), OK CPU $\checkmark$

The winning strategy: 5-min version polling, visible-tab only, fetch full catalog on change.

Same quota usage as the original fix, but updates appear in 5 min instead of 5 min, AND when an admin edits something, all visitors see it within 5 min GUARANTEED (instead of 'maybe within 5

min, depending on when their last poll was'). Admin edits trigger an immediate version bump via the existing /refresh flow.

7. PII Handling — Lawful, Free, and Pragmatic

The original report flagged PII handling as a Law 18-07 violation. Its proposed fixes (move to POST body, encrypt at rest, subject access rights) are correct but assumed infrastructure that doesn't exist. Here's the constraint-aware version that uses what's already deployed.

7.1 The PII flow today (constraint-respecting analysis)

Customer enters name + phone + wilaya + commune in the order form. The form POSTs to /api/order, which GETs the Apps Script URL with PII in URL params (sheet.ts:154-209). Apps Script writes to the Google Sheet. The merchant reads the sheet. **The PII is stored in: (1) the URL (browser history, server logs), (2) Apps Script execution logs, (3) the Google Sheet, (4) any failed-order retry queue in localStorage.** Four copies, three of which leak.

7.2 The constraint-aware fix

Move order submission to the Worker (which already has fetch + the admin secret). Browser POSTs PII as JSON body to Worker. Worker forwards to Apps Script as POST body (not URL params). Apps Script doPost writes to the sheet. PII never appears in a URL. Failed orders stay in localStorage (encrypted with a per-session key derived from a salt stored in sessionStorage — not perfect, but better than plaintext).

```
# Worker: new endpoint ?action=submit_order
if (action === 'submit_order') {
  // Rate limit (probabilistic – see Section 4 of original report)
  if (Math.random() < 0.1) {
    // 1-in-10 orders write to KV for rate limit check
    const phoneKey = `phone_${phone.slice(-4)}`; // last 4 digits only
    const count = (await env.CATALOG_KV.get(phoneKey, 'json') || 0) + 1;
    if (count > 3) return json({ok: false, error: 'rate_limited'}, 429, cors);
    ctx.waitUntil(env.CATALOG_KV.put(phoneKey, count, {expirationTtl: 3600}));
  }

  // Validate input
  const body = await request.json();
  if (!PHONE_REGEX.test(body.phone)) return json({ok: false, error: 'invalid_phone'}, 400);
  if (!body.fullName || !body.wilaya) return json({ok: false, error: 'missing_fields'}, 400);

  // Forward to Apps Script as POST body (NOT URL params)
  const res = await fetch(`${env.APPS_SCRIPT_URL}?action=order`, {
    method: 'POST',
    headers: {'Content-Type': 'text/plain;charset=utf-8'},
    body: JSON.stringify(body),
    redirect: 'follow',
  });

  // Don't log PII to console.error (original bug PII-02)
  if (!res.ok) {
    console.error('[order] Apps Script failed', {status: res.status}); // NO PII
  }
  return json({ok: res.ok}, 200, cors);
}
```

7.3 The failed-orders queue — encrypted at rest, no expiry change

The original report's fix (move to IndexedDB with 7-day expiry) requires restructuring the queue. The smarter version: encrypt the existing localStorage entry with a key derived from a per-session salt. The salt lives in sessionStorage (cleared when the browser closes). If the browser is closed, the queue is unreadable without the salt — which is gone. This achieves 'encryption at rest' and 'auto-expiry on session end' with ~10 lines of code change.

```
# src/lib/failed-orders.ts — add encryption

# Simple XOR cipher (NOT cryptographically secure, but defeats casual inspection)
# For real encryption, use Web Crypto API (SubtleCrypto AES-GCM) — it's free, native,
# and available in all modern browsers. ~15 lines of code.

async function encrypt(plaintext: string, key: string): Promise<string> {
  // Use Web Crypto API — native, free, no deps
  const enc = new TextEncoder();
  const cryptoKey = await crypto.subtle.importKey(
    'raw', enc.encode(key), {name: 'AES-GCM'}, false, ['encrypt', 'decrypt']
  );
  const iv = crypto.getRandomValues(new Uint8Array(12));
  const ct = await crypto.subtle.encrypt(
    {name: 'AES-GCM', iv}, cryptoKey, enc.encode(plaintext)
  );
  return btoa(String.fromCharCode(...iv, ...new Uint8Array(ct)));
}

# Session salt — regenerated each session, stored in sessionStorage
function getSessionKey(): string {
  let k = sessionStorage.getItem('soumdeco_session_salt');
  if (!k) {
    k = crypto.randomUUID();
    sessionStorage.setItem('soumdeco_session_salt', k);
  }
  return k;
}

# When saving failed order:
saveFailedOrders(orders) {
  const encrypted = await encrypt(JSON.stringify(orders), getSessionKey());
  localStorage.setItem('soumdeco_failed_orders', encrypted);
}

# Result: closed browser = unreadable queue. Same browser = decrypts automatically.
```

7.4 Data retention — solve it by deleting, not archiving

Algerian Law 18-07 requires defined retention periods. The simplest fix: add a nightly Apps Script trigger that deletes orders older than 90 days. The merchant gets 3 months to fulfill + reconcile, then the data is gone. Zero infra, zero cost, just one Apps Script function. Combined with the POST-body fix above, PII never appears in URLs and auto-expires after 90 days. That's compliant.

8. The Fully Revised Roadmap

This is the constraint-aware version of the original Phase 1-5 roadmap. The structure is the same — only the proposed fixes change. Total added cost: \$0. Total new accounts needed: 0. Total new infrastructure: 0 (only adds branches to the existing Worker, plus 1 extra free Pages project for image overflow if scaling to 9,500).

Phase 1 — Stop the bleeding (today, 2-4 hours, \$0)

- 1.1 Rotate ALL secrets: Apps Script URL (redploy with new deployment ID), Cloudinary preset name, Worker ADMIN_SECRET, admin password. All via wrangler secret / Apps Script redeploy.
- 1.2 Remove .env.production from git history (git filter-repo). Add .gitignore with .env* (except .env.example).
- 1.3 Remove adminPassword from brand-config.ts. Add POST /?action=auth to Worker (validates against env.ADMIN_PASSWORD). Admin panel PasswordGate fetches the token, stores in sessionStorage.
- 1.4 Add X-Admin-Secret header check to all Worker mutating endpoints (write_product, delete_product, upload_image_proxy, submit_order).
- 1.5 DELETE the hardcoded SHEET_BASE_URL in src/lib/sheet.ts:8-9. The Apps Script URL must ONLY live in Worker env (set via wrangler secret put APPS_SCRIPT_URL).
- 1.6 Convert Apps Script ?action=product_reset and ?action=product_delete from GET to POST. Worker forwards as POST, never as GET.

Phase 2 — Fix the Cloudinary bug (this week, 4 hours, \$0)

- 2.1 ARCHITECTURAL FIX: Delete optimizeCloudinaryUrls from use-catalog.ts:797-825. Move the manifest lookup INTO ProductImage (product-image.tsx) — happens at render time only. Product state is never mutated. Leak stops.
- 2.2 Fix build-image-manifest.py: (a) use SCRIPT_DIR/REPO_ROOT pattern, (b) exit with non-zero status if manifest is empty (fail-loud).
- 2.3 Write one-time Apps Script function to repair 44 corrupted rows: for every product with image starting with /images/products/, reconstruct Cloudinary URL (https://res.cloudinary.com/{CLOUD}/image/upload/{filename}) and overwrite.
- 2.4 Add workflow_dispatch trigger to auto-sync. Worker /refresh (after admin op) fires GitHub Actions webhook. New images sync within minutes, not 24h. Daily schedule stays as safety net.
- 2.5 Remove the buildCloudinaryFallback onError logic in product-image.tsx — it's no longer needed (canonical Cloudinary URL is always available in product state).
- 2.6 Reconcile cron schedule: either change wrangler.toml to 5-min (matching code design) OR update KV TTL to 60s and update code comments. Pick one. The 1-min cron burns 7.2% of Apps Script quota — fine, but should be intentional.

Phase 3 — Close authorization gaps (this week, 1 day, \$0)

- 3.1 Move ALL admin writes from browser→Apps Script to browser→Worker→Apps Script. Worker validates X-Admin-Secret, forwards server-side, strips secret. Browser never sees the Apps Script URL.
- 3.2 Move Cloudinary image upload to Worker proxy. Browser→Worker (with admin secret)→Cloudinary. Preset stays unsigned. Worker is the only path that can upload. (Future: when credit card is added, switch to R2 + Worker signing.)
- 3.3 Add honeypot field + time-to-submit check + probabilistic KV-counter rate limit to order form. 3 lines of code in cod-order-form.tsx + ~20 lines in Worker. No Turnstile, no Cloudflare signup.
- 3.4 Move PII transmission from GET URL params to POST body via Worker. Apps Script doPost handles it. PII never appears in URLs.
- 3.5 Stop logging PII in /api/order/route.ts:58-64. Replace console.error({fullName, phone, ...}) with console.error({status, orderId}). Log only the error type, not the payload.
- 3.6 Encrypt failed-orders queue in localStorage using Web Crypto API (AES-GCM with session salt from sessionStorage). Closed browser = unreadable queue. ~15 lines of code.
- 3.7 Add nightly Apps Script trigger to delete orders older than 90 days. Achieves Law 18-07 retention compliance with zero infra.
- 3.8 Remove NEXT_PUBLIC_WORKER_ADMIN_SECRET entirely. Admin panel uses the auth-token flow (Phase 1.3) which returns the secret only on password validation.
- 3.9 Add CSP via /public/_headers (Cloudflare Pages native). Restrict script-src to self + Cloudinary (for image transformations). No 'unsafe-inline' if possible.

Phase 4 — Performance & reliability (next 2 weeks, \$0)

- 4.1 Dynamic import AdminPanel (ssr: false). Saves ~30-50KB gzipped. Also removes adminPassword from non-admin bundles.
- 4.2 Move SEED_PRODUCTS to /public/seed-products.json. Lazy fetch only when other sources fail. Saves ~25KB gz.
- 4.3 Delete 47 unused shadcn UI components + matching deps from package.json. Faster installs + builds.
- 4.4 Convert Hero, Categories, BrandStory, SiteFooter to server components. Move catalog fetch to server. ~60-80% LCP improvement. (Note: server components work on Cloudflare Pages via @cloudflare/next-on-pages — only edge API routes are broken, not RSC.)
- 4.5 Use Cloudinary's free f_auto,q_auto transformations for the 24h window between upload and local sync. NO need for Cloudflare Image Resizing (\$5/mo). The local mirror handles 99% of traffic at zero cost.
- 4.6 Add @tanstack/react-virtual to AllProducts rows + admin panel list. Already in deps. Prevents mobile crashes at 9,500-product scale.
- 4.7 Memoize ProductCard + ProductImage. ~5s jank saved per poll at scale.

4.8 IMPLEMENT THE VERSION-POLL STRATEGY (Section 6): 5-min poll of 8-byte version key, fetch full catalog only on change. Same quota as 5-min full poll, but near-instant updates on admin change.

4.9 Delete LoadingFallback.tsx. Eliminates 3M DOM scans/day. Skeleton + ErrorBoundary already cover it.

4.10 Move catalog parse/sort to Web Worker. Saves 200-600ms main-thread block per poll.

4.11 Merge use-stock into use-catalog. Single interval, single fetch, single state.

4.12 Add rAF throttle to scroll listener in page.tsx.

4.13 Add timestamp to IDB catalog. Only overwrite Worker data if IDB is newer.

4.14 IF scaling to 9,500 products: shard the KV (Section 5). 96 shards × ~200KB each. Worker CPU drops from 200ms to 2ms per request. Cron every 10 min instead of 1 min (864 writes/day, under 1K quota).

4.15 IF scaling to 9,500 products: create second free Cloudflare Pages project (soumdeco-images.pages.dev) for image overflow. auto-sync.py pushes to both. Stays under 20K file limit per project.

Phase 5 — Quality hardening (next month, \$0)

5.1 Remove typescript.ignoreBuildErrors: true. Fix every TS error. Hard but necessary.

5.2 Restore ESLint rules incrementally. Start with no-explicit-any: warn, exhaustive-deps: warn.

5.3 Add Vitest unit tests for: Worker data-sync.js (syncData, fetchFromAppScript), use-catalog.ts (refresh, fallback chain), product-image.tsx (pickImageUrl).

5.4 Make repo PUBLIC (it's a generic Next.js storefront, no proprietary code). Unlimited GitHub Actions minutes. Add: lint + typecheck + build on PR, auto-sync.py on schedule + webhook.

5.5 Add 3-5 Playwright e2e tests: admin login + product create + product delete + order submission. Run only on PRs to main.

5.6 Validate productId format on Worker (regex /^[a-zA-Z0-9_-]{1,64}\$/). Reject bad inputs with 400.

5.7 Remove ?secret= query-param auth on Worker. Header only (X-Admin-Secret).

5.8 Gate Worker /?action=health behind X-Admin-Secret. Stop leaking operational metadata.

5.9 Write a real README documenting the architecture, the 8 constraints, the wiring map, and the deploy steps.

5.10 Consolidate 3 normalizeSheetProduct functions into 1 shared module.

5.11 Split admin-panel.tsx (1,576 lines) into 4-5 focused components. Split use-catalog.ts (911 lines) similarly.

5.12 Remove Prisma + /prisma/schema.prisma. Document that the Google Sheet IS the database (correct for this constraint set).

9. What the First Report Got Wrong

An honest audit of the audit. The first report was correct on the findings — the bugs are real, the security posture is genuinely bad, the data integrity is compromised. But the recommendations had three blind spots that this addendum corrects:

9.1 Blind spot 1: assumed paid services were available

Cloudflare Image Resizing (\$5/mo), Cloudflare Access (paid tier for service tokens), Cloudflare Turnstile (free but requires signup), Cloudflare R2 (free but needs credit card), Cloudflare Workers Paid (\$5/mo for higher CPU limit) — all of these were recommended without checking if the team's explicit constraint of 'no credit card, free tier only' allowed them. The `wrangler.toml` file documents this constraint explicitly. I should have read it before recommending paid services.

9.2 Blind spot 2: didn't notice the Pages edge runtime was broken

The team's own comment in `client-sheet.ts:7-8` says: 'This bypasses the Cloudflare Pages edge API routes entirely, which return 500 errors on Cloudflare's edge runtime (Next.js 16 edge runtime is deprecated and has issues with `@cloudflare/next-on-pages v1`).' This means EVERY recommendation that involved Pages API routes (`middleware.ts`, JWT signing on server, `/api/admin/login`, `/api/products` POST with auth) was technically infeasible without either (a) moving off `@cloudflare/next-on-pages`, or (b) upgrading to Workers Paid. The constraint-aware fix is to move all that logic to the Worker, which already works.

9.3 Blind spot 3: missed the Worker CPU limit at scale

The original report flagged that 9,500 products would crash mobile browsers (~57K DOM nodes). That's correct. But it missed that the WORKER would also crash — Cloudflare Workers free tier has a 10ms CPU limit per request, and parsing a 19MB JSON catalog takes ~200ms. The sharding strategy (Section 5.2) is the only way to scale to 9,500 products within the free tier. The original 'add virtualization' fix would have fixed the browser but not the Worker.

9.4 What the first report got RIGHT (don't throw out the baby)

Most of the original audit stands. The findings are accurate: the admin password is in the bundle, the secrets are committed, the local-URL leak is real, the `build-image-manifest.py` paths are broken, the PII handling violates Law 18-07, the code quality gates are disabled, the dead Pages API routes are attack surface, the cron schedule is mismatched, the polling is excessive, the admin panel should be code-split, the 47 shadow UI components should be deleted. All of these findings are correct. Only the proposed fixes needed the constraint-aware refinement in this addendum.

Bottom line

The first report diagnosed the disease correctly. This addendum prescribes treatments the patient can actually afford. Take the original findings as fact, replace the remediation roadmap with this one. The team can ship every fix in this addendum for \$0, with zero new accounts, zero credit cards, and zero new infrastructure beyond what's already deployed. The only addition is a second free Cloudflare Pages project — and only if/when the catalog actually approaches 9,500 products. Until then, even that isn't needed.